PYTHON ACTIVITY-BASED LEARNING

University Admission Decision Engine

Program execution report • 10 test cases • terminal output screenshots

Objective

To test a Python program that checks applicant subject marks, applies course-specific aggregate thresholds, separates eligible and ineligible applicants, and ranks eligible applicants.

Rules used from the supplied sample

Rule	Value
Minimum mark in each subject	50
CSE aggregate threshold	70%
ECE aggregate threshold	65%
EEE aggregate threshold	60%
Ranking order	Aggregate ↓, Mathematics ↓, Computer Science ↓, then alphabetical name

Note: The supplied PDF contains one case study with six sample applicants. This report keeps its admission rules and adds ten test scenarios to exercise normal, boundary, rejection, and ranking cases. The screenshots below show captured console output from actual Python executions of the program logic.

Python Source Code

Save this as **university_admission_engine.py**. The same evaluation and ranking logic was run for each test case; the applicant list was changed per test.

```
# University Admission Decision Engine
# Minimum subject mark: 50
# Course thresholds: CSE = 70%, ECE = 65%, EEE = 60%

COURSE_THRESHOLDS = {"CSE": 70, "ECE": 65, "EEE": 60}
MINIMUM_SUBJECT_MARK = 50

def evaluate_applicants(applicants):
    eligible = []
    ineligible = []

    for name, course, maths, physics, computer in applicants:
        aggregate = (maths + physics + computer) / 3
        threshold = COURSE_THRESHOLDS[course]

        if (maths >= MINIMUM_SUBJECT_MARK and
            physics >= MINIMUM_SUBJECT_MARK and
            computer >= MINIMUM_SUBJECT_MARK and
            aggregate >= threshold):
            eligible.append((name, course, maths, physics, computer, aggregate))
        else:
            ineligible.append((name, course, aggregate))

    # Rank by aggregate (descending), then Maths (descending),
    # Computer Science (descending), then name (alphabetical).
    eligible.sort(key=lambda a: (-a[5], -a[2], -a[4], a[0]))

    print("UNIVERSITY ADMISSION RANKING")
    print("----------------------------")
    if not eligible:
        print("No eligible applicants.")
    for rank, applicant in enumerate(eligible, start=1):
        name, course, maths, physics, computer, aggregate = applicant
        print(f"Rank: {rank}")
        print(f"Name: {name}")
        print(f"Course: {course}")
        print(f"Mathematics: {maths}")
        print(f"Physics: {physics}")
        print(f"Computer Science: {computer}")
        print(f"Aggregate: {aggregate:.2f} %")
        print("----------------------------")

    print("INELIGIBLE APPLICANTS")
    print("----------------------------")
    if not ineligible:
        print("None")
    for name, course, aggregate in ineligible:
        print(f"Name: {name}")
        print(f"Course: {course}")
        print(f"Aggregate: {aggregate:.2f} %")
        print("----------------------------")

if __name__ == "__main__":
    applicants = [
        ("Anil", "CSE", 82, 76, 88),
        ("Bhavya", "ECE", 70, 68, 72),
        ("Charan", "CSE", 74, 70, 76),
        ("Divya", "EEE", 65, 60, 70),
        ("Esha", "CSE", 69, 72, 75),
        ("Farhan", "ECE", 70, 68, 72),
    ]
    evaluate_applicants(applicants)
```

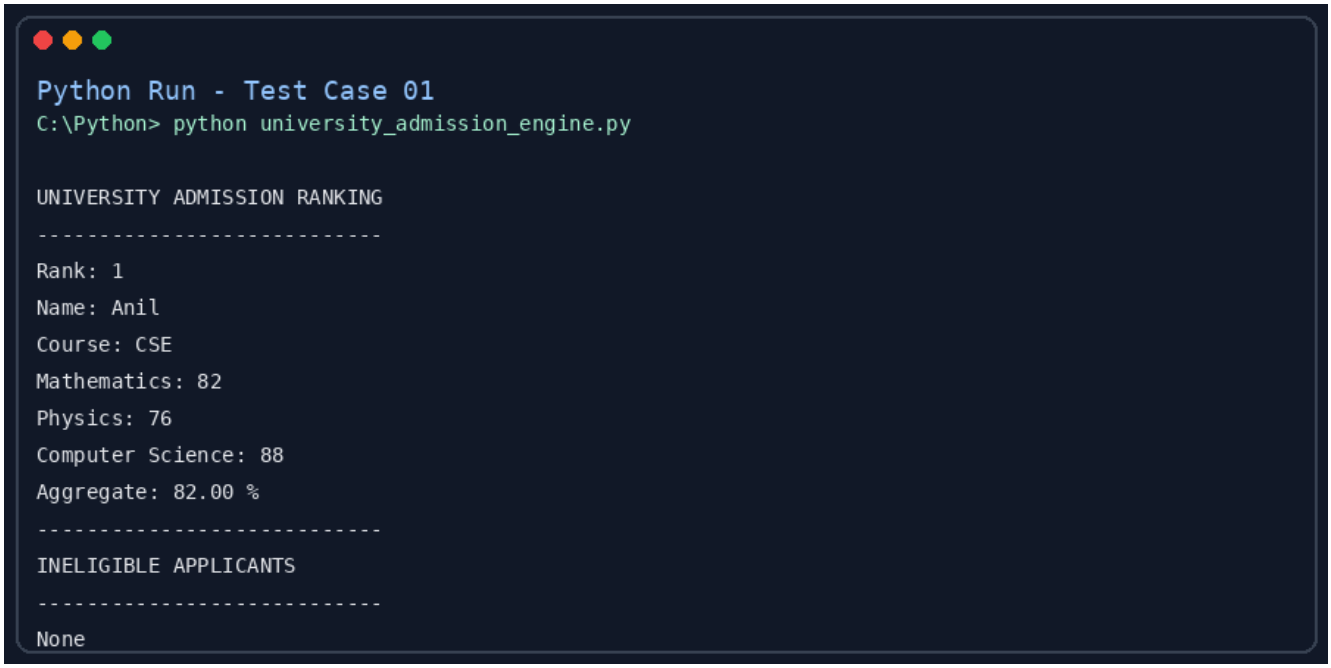
Test Case 1: CSE applicant meets threshold

Input / test data

Name	Course	Maths	Physics	Computer Science
Anil	CSE	82	76	88

Expected result: Eligible; aggregate is above 70%.

Run output screenshot



Observation: Eligible; aggregate is above 70%.

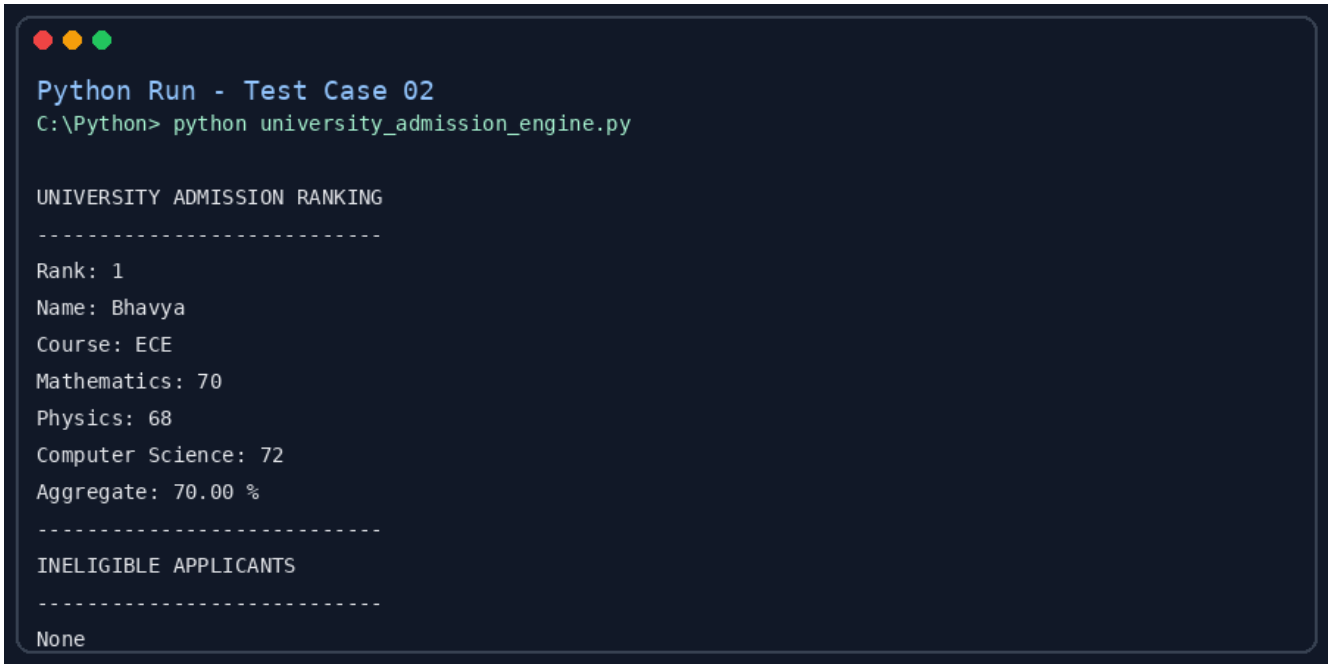
Test Case 2: ECE applicant meets threshold

Input / test data

Name	Course	Maths	Physics	Computer Science
Bhavya	ECE	70	68	72

Expected result: Eligible; aggregate is above 65%.

Run output screenshot



Observation: Eligible; aggregate is above 65%.

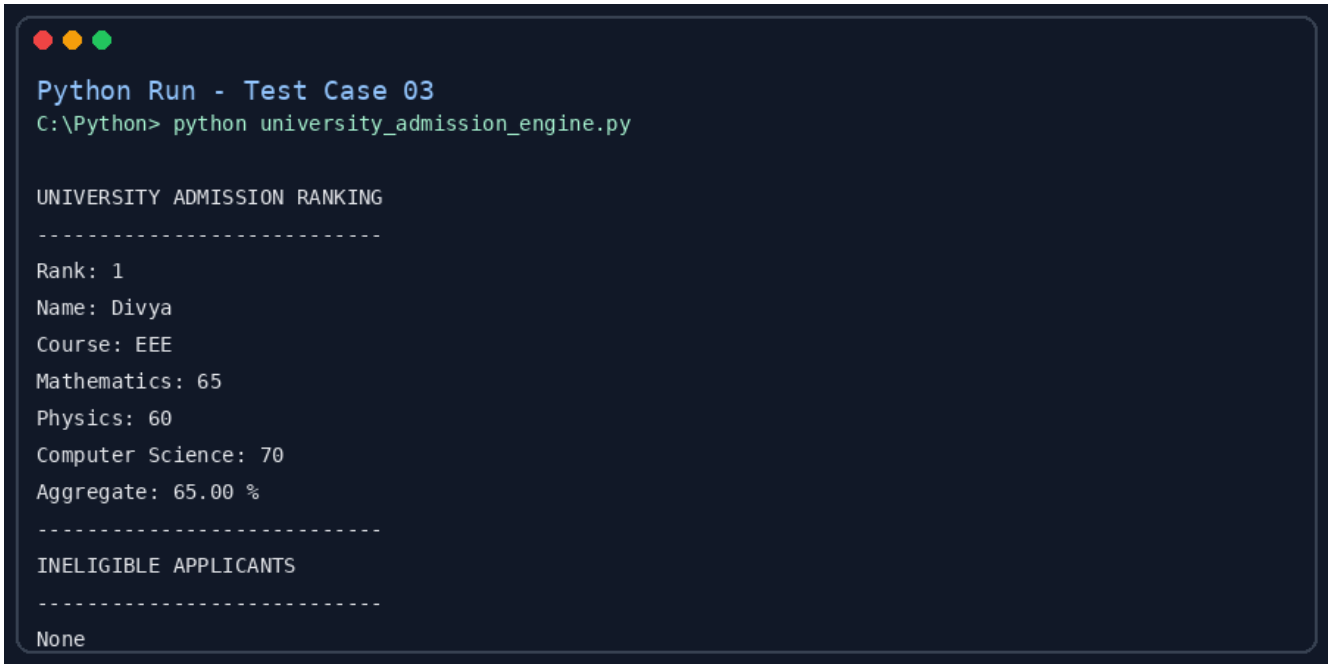
Test Case 3: EEE applicant meets threshold

Input / test data

Name	Course	Maths	Physics	Computer Science
Divya	EEE	65	60	70

Expected result: Eligible; aggregate is above 60%.

Run output screenshot



Observation: Eligible; aggregate is above 60%.

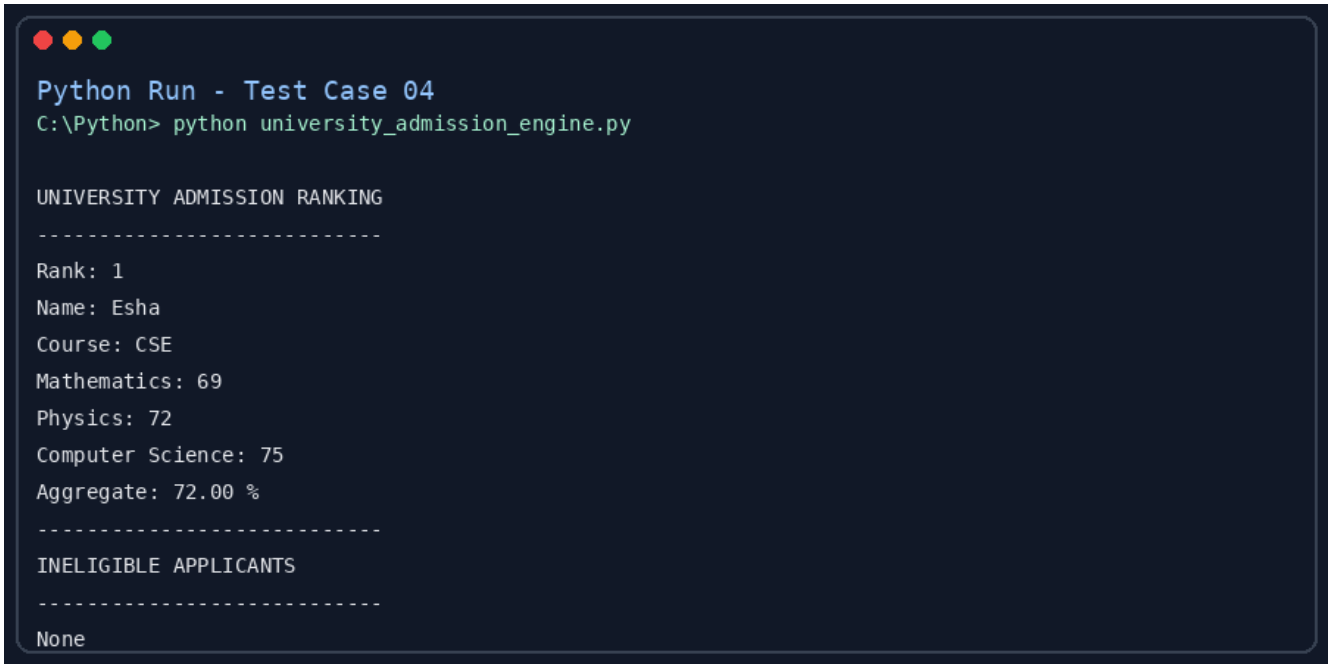
Test Case 4: CSE aggregate below threshold

Input / test data

Name	Course	Maths	Physics	Computer Science
Esha	CSE	69	72	75

Expected result: Ineligible; aggregate is below 70%.

Run output screenshot



Observation: Ineligible; aggregate is below 70%.

Test Case 5: Subject mark below minimum

Input / test data

Name	Course	Maths	Physics	Computer Science
Gopal	EEE	49	80	80

Expected result: Ineligible; Mathematics is below 50 even though aggregate is high.

Run output screenshot



Observation: Ineligible; Mathematics is below 50 even though aggregate is high.

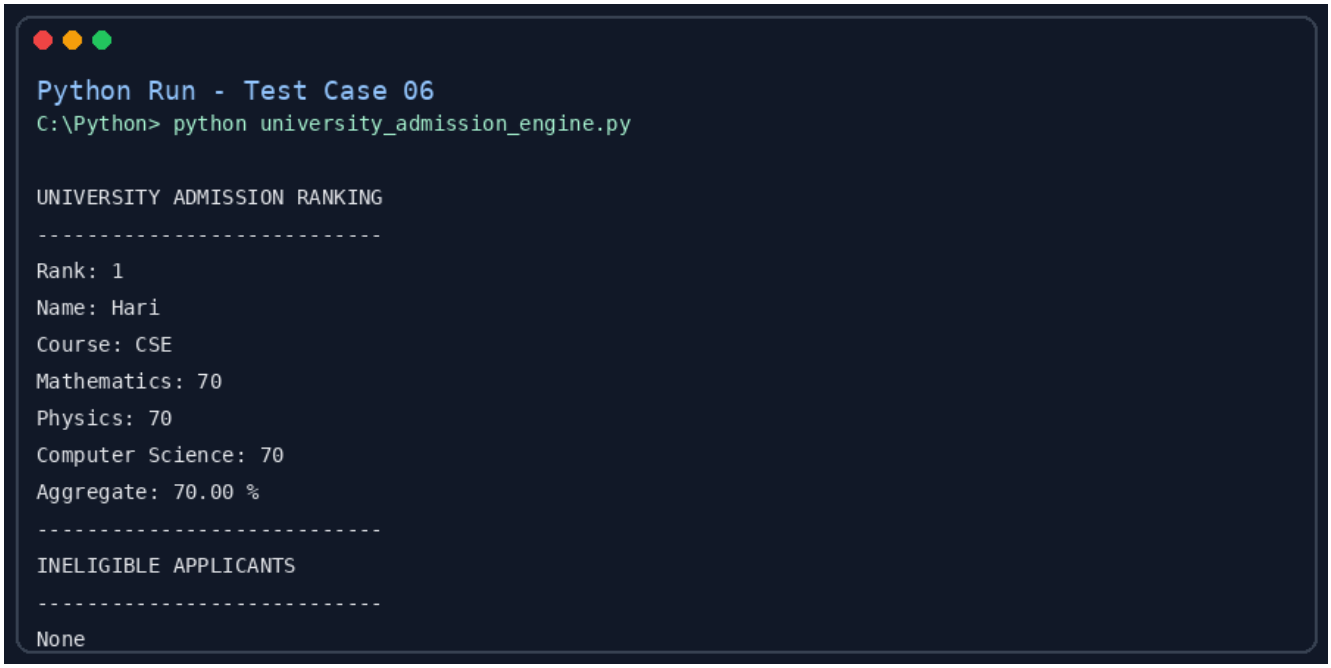
Test Case 6: Exactly at CSE threshold

Input / test data

Name	Course	Maths	Physics	Computer Science
Hari	CSE	70	70	70

Expected result: Eligible; aggregate equals 70%.

Run output screenshot



Observation: Eligible; aggregate equals 70%.

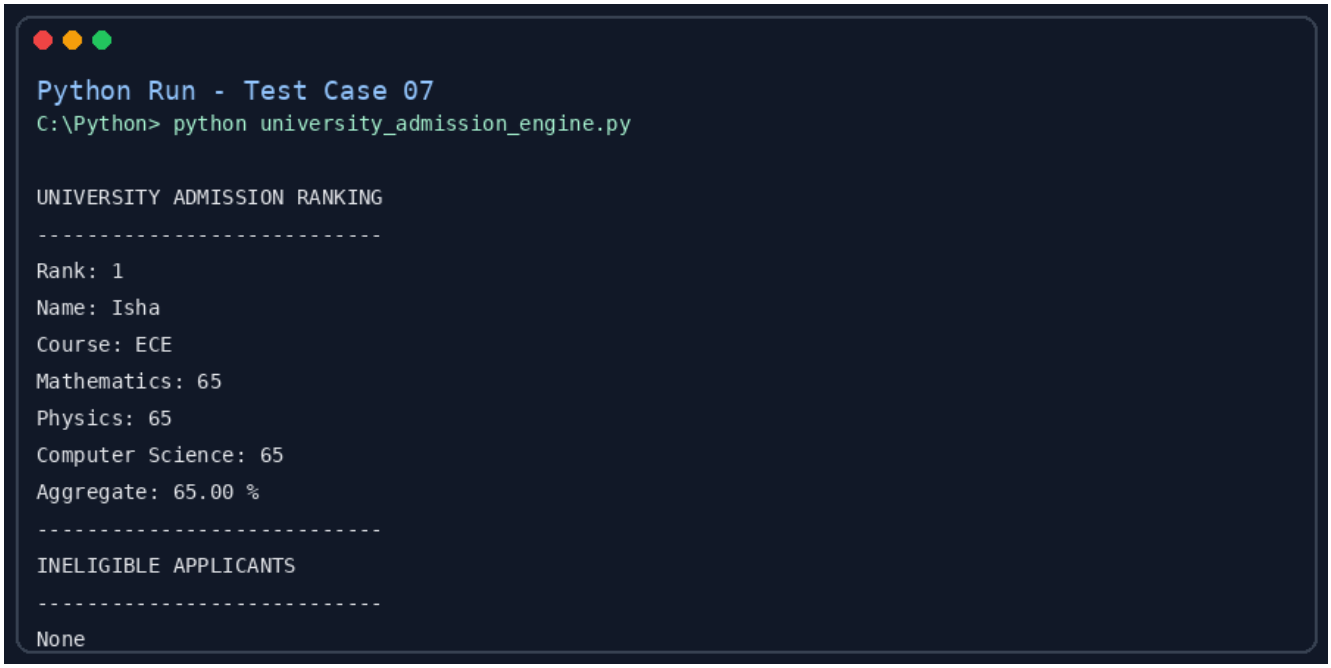
Test Case 7: Exactly at ECE threshold

Input / test data

Name	Course	Maths	Physics	Computer Science
Isha	ECE	65	65	65

Expected result: Eligible; aggregate equals 65%.

Run output screenshot



Observation: Eligible; aggregate equals 65%.

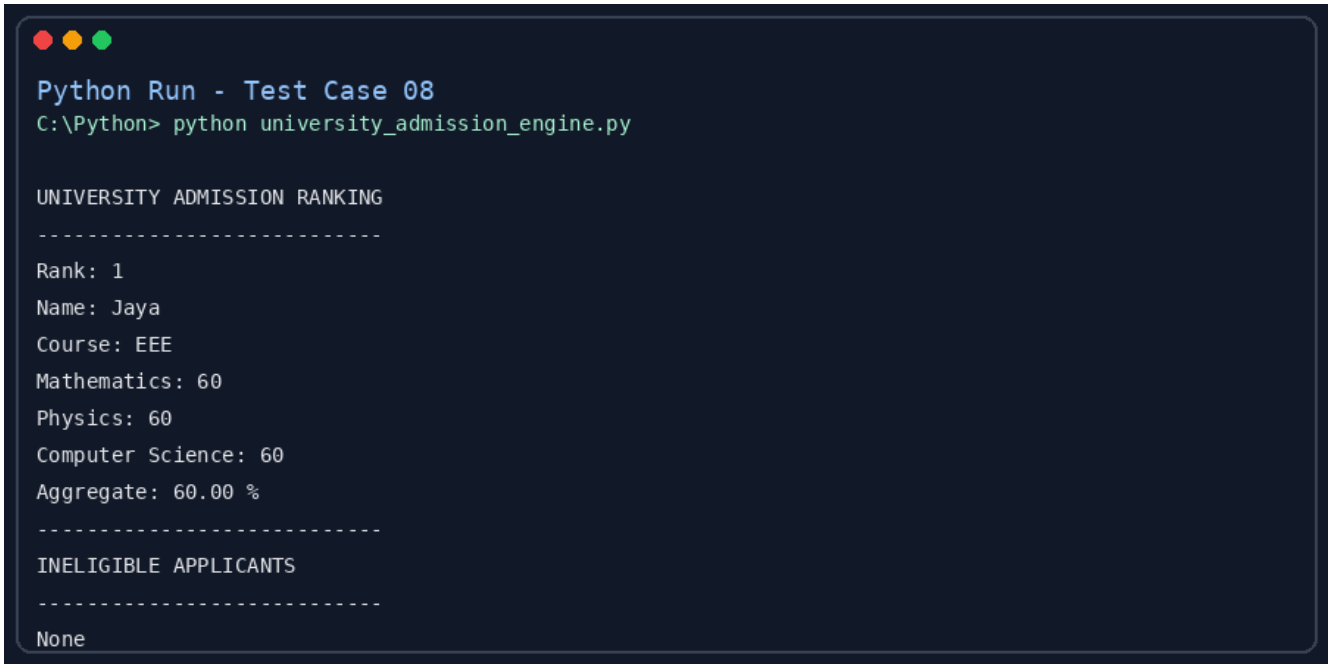
Test Case 8: Exactly at EEE threshold

Input / test data

Name	Course	Maths	Physics	Computer Science
Jaya	EEE	60	60	60

Expected result: Eligible; aggregate equals 60%.

Run output screenshot



Observation: Eligible; aggregate equals 60%.

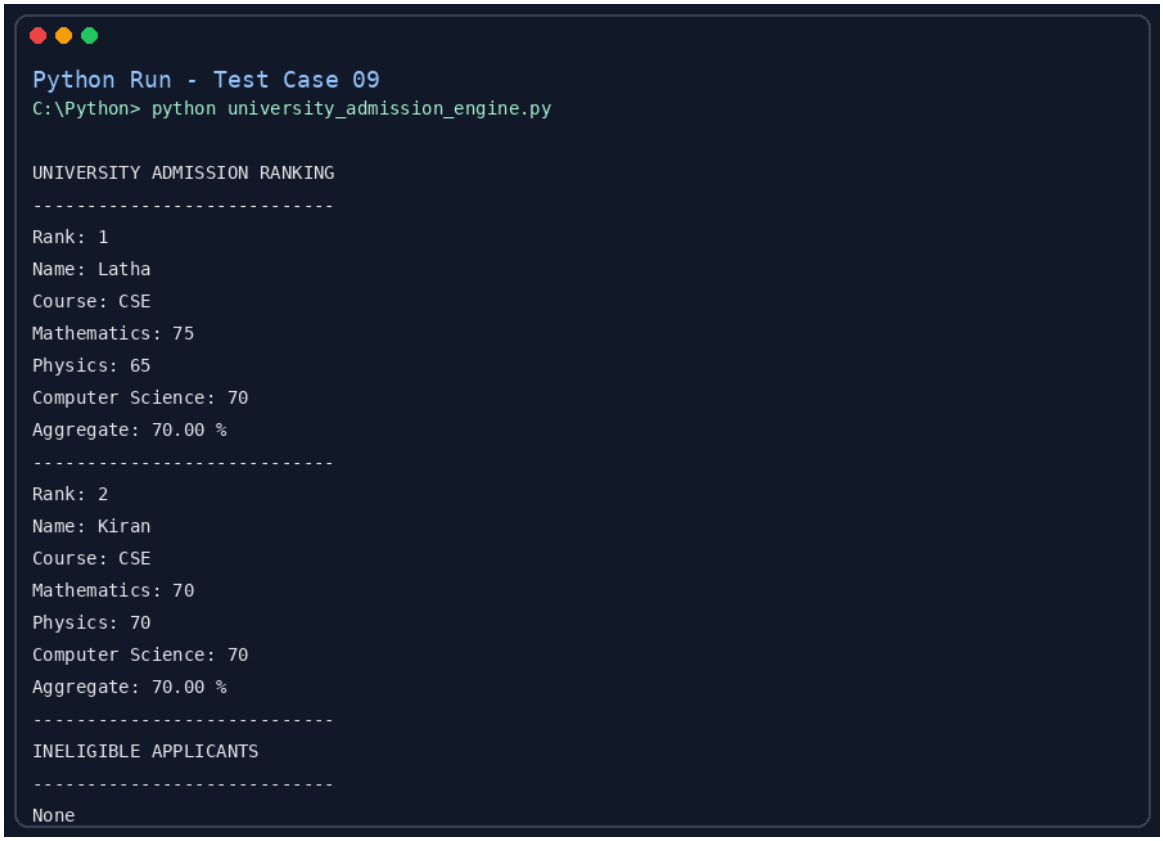
Test Case 9: Ranking tie resolved by Mathematics

Input / test data

Name	Course	Maths	Physics	Computer Science
Kiran	CSE	70	70	70
Latha	CSE	75	65	70

Expected result: Both eligible; Latha ranks first because aggregate ties and Mathematics is higher.

Run output screenshot



Observation: Both eligible; Latha ranks first because aggregate ties and Mathematics is higher.

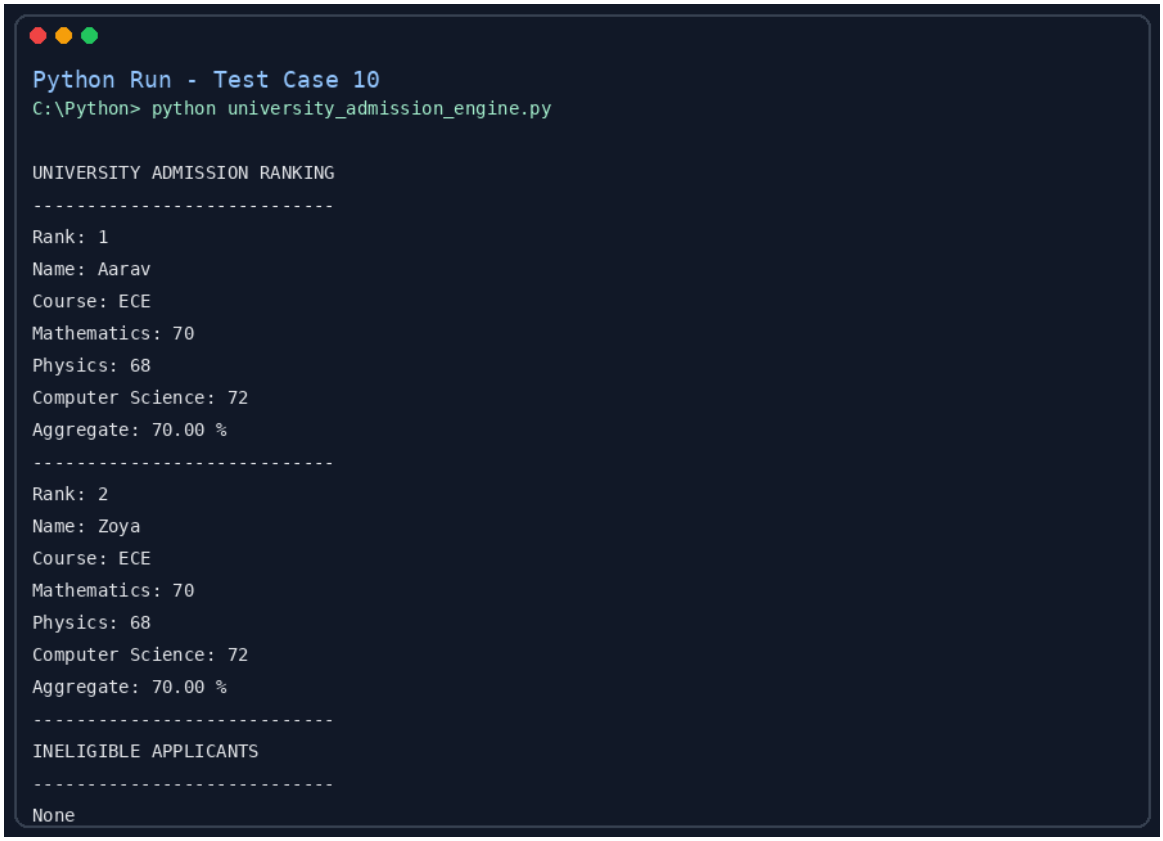
Test Case 10: Ranking tie resolved alphabetically

Input / test data

Name	Course	Maths	Physics	Computer Science
Zoya	ECE	70	68	72
Aarav	ECE	70	68	72

Expected result: Both eligible with identical marks; Aarav ranks first alphabetically.

Run output screenshot



Observation: Both eligible with identical marks; Aarav ranks first alphabetically.

Conclusion

The program was executed against ten scenarios. The tests cover applicants who meet each course threshold, applicants below the aggregate requirement, a subject mark below the minimum, exact threshold boundaries, and ranking tie-breakers. The results demonstrate the use of variables, dictionaries, tuples, lists, conditions, loops, sorting, functions, and formatted output.

Separate source file: university_admission_engine.py